

Week 12: Fine-Tuning — What, Why & Continued Pre-Training from Scratch

When the prompt isn't enough, we change the model itself.

WEEK

12

DATE

04 APR 2026

FOCUS

FINE-TUNING, LORA & CPT

We've spent weeks learning to talk to models (Prompting), give them external knowledge (RAG), and hand them tools (Agents). But what happens when none of that is enough? What if the model doesn't speak your domain's language, can't hold a specific output format, or is simply too large and slow for production? This week, we cross the final frontier: we stop adapting our inputs and start adapting the model's **weights** directly. We dissect the full post-training pipeline, break down the "Memory Math" that makes naive fine-tuning impossible on consumer hardware, discover the elegant mathematical trick behind LoRA, and build a complete Continued Pre-Training pipeline on real SEC 10-K filings — watching a 135M-parameter model go from confused gibberish to fluent financial language in under 16 minutes on a free T4 GPU.

Quick Links & Resources

Resource	Link
 Lecture Slides	Google Drive
 Colab Notebook	Google Colab (Local Copy: Codes/cLass_9a_f.ipynb)

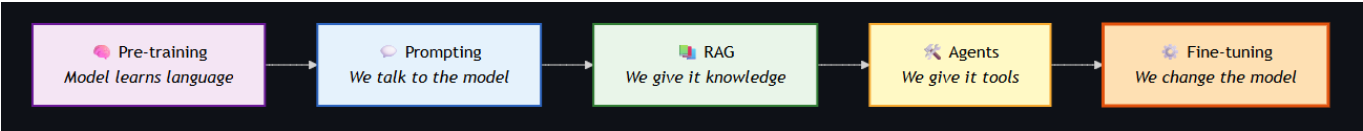
Table of Contents

#	Section	Core Concept
	The Journey So Far	How we got here: the progressive ladder from pre-training to fine-tuning.
1	What is Fine-Tuning?	Same mechanism as pre-training, vastly different scale. The university graduate analogy.
2	Why Fine-Tune?	The exact failure modes of prompting and RAG that fine-tuning solves.
3	The Decision Framework	A diagnostic table: CPT vs SFT vs RLHF vs RLVR — matching problem to solution.
4	The Post-Training Pipeline	The exact sequence from "next-token predictor" → helpful, aligned, reasoning assistant.
5	What's Inside a Model	A tour of Attention blocks, FFN blocks, and why they're all just weight matrices.

#	Section	Core Concept
6	Full Fine-Tuning & The Memory Math	Why updating every weight requires 6-8x model size in VRAM, with real examples.
7	LoRA — The Breakthrough	The key insight, the choir analogy, the exact math: 64x fewer parameters.
8	LoRA Visually & At Inference	Dual-path architecture: frozen original + trainable adapters → zero-cost merge.
9	Memory Savings & Hyperparameters	From 64 GB → 4.2 GB. Rank, Alpha, Target Modules, and the CPT-specific gotcha.
10	Quantization (QLoRA)	Float32 → NF4: compressing frozen weights 8x with almost no quality loss.
111	Unsloth — Same Math, Faster	Custom CUDA kernels, gradient checkpointing, packing, and NEFTune.
112	The Training Loop Internals	Tokenize → Forward → Loss → Backward → Update. Same loop for pre-training, CPT, SFT.
113	Catastrophic Forgetting	Why greedy gradients erase general knowledge, and 5 solutions to prevent it.
114	Hands-On: CPT with SEC 10-K Filings	Full pipeline: data prep, LoRA config, training signals, and real before/after outputs.
115	The Forgetting Experiment	Run A (100% domain) vs Run B (80/20 mix). Quantifying the forgetting trade-off.
116	The Limits of CPT	CPT teaches SOUND, SFT teaches RESPOND. Why a fluent model still can't answer questions.
117	Key Takeaways	The 7 core truths of fine-tuning, distilled.

The Journey So Far

Every technique we've learned builds on the last. Fine-tuning is the final layer — the one where we stop adapting our *approach* and start adapting the *model itself*.



Stage	What Changes	What Stays Fixed
Pre-training	Everything (model learns from scratch)	Nothing
Prompting	The input text we craft	The model weights
RAG	The context we inject before the prompt	The model weights

Stage	What Changes	What Stays Fixed
Agents	The tools the model can call	The model weights
Fine-tuning	The model weights themselves	The architecture

1 What is Fine-Tuning?

Fine-tuning uses the **exact same learning mechanism** as pre-training — gradient descent on a next-token prediction loss — but at a fundamentally different scale and with targeted data.

Dimension	Pre-training	Fine-tuning
Goal	Learn language, facts, and reasoning from the entire internet	Adapt that knowledge for a specific domain or task
Data	Trillions of tokens (Common Crawl, Wikipedia, Books, Code)	Thousands to millions of tokens (your domain data)
Duration	Weeks to months on thousands of GPUs	Hours on a single GPU
Cost	\$\$\$\$\$\$ (millions of dollars)	\$ (dollars to hundreds)

Same mechanism. Different scale. Different data.

🎓 The University Graduate Analogy

A pre-trained model is like a **university graduate**. They've already spent years learning how to read, write, reason, and understand the world. You don't need to teach them from scratch. Fine-tuning is simply **company onboarding** — training them for your specific job.

You wouldn't re-teach them the alphabet. You'd hand them your company's style guide, your codebase, your domain terminology, and let them adapt.

2 Why Fine-Tune? (When Prompting & RAG Fail)

We already have powerful tools. When exactly do they break down?

✗ Prompting Fails When...	✗ RAG Fails When...
The model doesn't know your domain's specific jargon or terminology (e.g., "EBITDA", "subordinated debentures").	The issue isn't missing knowledge — it's wrong behavior or style .
You need a hyper-specific output format or tone applied consistently across thousands of requests.	Latency matters and you can't afford the retrieval round-trip (no retrieval step).
You want to deploy a smaller, faster, cheaper model that matches a larger model's quality on your task.	The model can't understand or synthesize the chunks it retrieves — the domain language itself is foreign.

✗ Prompting Fails When...

✗ RAG Fails When...

The desired behavior is too nuanced or complex to encode in a system prompt.

[!IMPORTANT] **The Key Insight:** RAG adds **knowledge**. Fine-tuning changes **behavior** and **style**. They solve fundamentally different problems. The best production systems combine both — a fine-tuned model that has been taught *how* to talk about your domain, augmented with RAG to provide *current* facts it hasn't memorized.

3 The Decision Framework

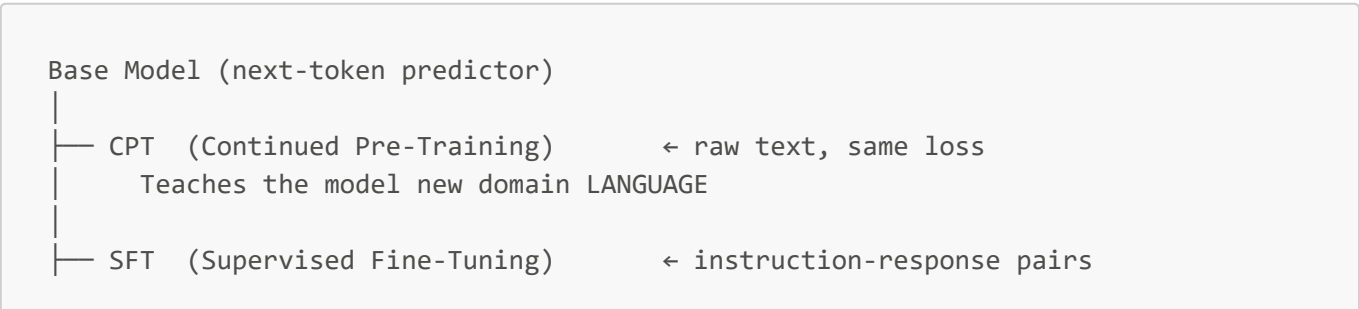
When your model isn't working, the first step is diagnosing the exact failure mode. Each failure maps to a specific intervention:

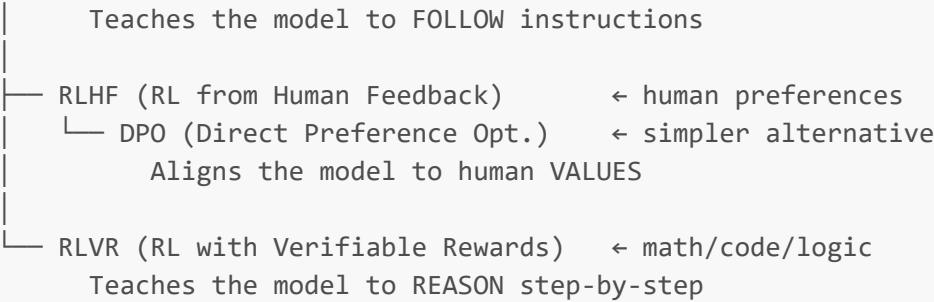
If your model...	Then use...	Full Name	What It Does
"...doesn't KNOW something"	RAG or CPT	Retrieval-Augmented Generation / Continued Pre-Training	Adds factual knowledge through external documents or domain-specific training data
"...doesn't DO something well"	SFT	Supervised Fine-Tuning	Teaches the model to follow specific instructions by training on input-output pairs
"...does it, but not how humans want"	RLHF / DPO	RL from Human Feedback / Direct Preference Optimization	Aligns outputs to human preferences ("which response is better?")
"...needs to reason better"	RLVR	RL with Verifiable Rewards	Trains logical/mathematical reasoning via problems with checkable answers

[!TIP] The most robust production systems don't pick just one. They **layer all of them** in sequence: CPT → SFT → RLHF/DPO → RLVR.

4 The Full Post-Training Pipeline

A base model fresh out of pre-training is just a "next-token predictor." It doesn't know it's supposed to be a helpful assistant. We transform it through a **multi-stage pipeline**, where each stage builds on the previous one:





Today: CPT (what the model *knows*). **Next session:** SFT (what the model *does*). **Then:** RLHF/DPO → RLVR (what the model *values*).

5 What's Actually Happening Inside a Model

To understand *why* fine-tuning runs into memory walls, we need to look at what a model physically is. A Transformer is a stack of identical layers. Each layer contains two major blocks, and every block is made of **weight matrices** — giant grids of numbers.

Block	Weight Matrices	What It Decides
Attention Block	Q_proj (W_q), K_proj (W_k), V_proj (W_v), O_proj (W_o)	"Which tokens should I pay attention to?"
FFN Block (Feed-Forward)	gate_proj (W_g), up_proj (W_up), down_proj (W_dn)	"What should I do with the information?" — This is where factual knowledge is stored .

Plus two special layers:

- **embed_tokens** (input): Converts token IDs → vectors. "What does this word mean?"
- **lm_head** (output): Converts vectors → token probabilities. "What word should come next?"

[!NOTE] **These are ALL weight matrices. Giant grids of numbers.** When we "train" a model, we are systematically updating these millions (or billions) of numerical entries. When we "fine-tune," we're doing the same thing — just on a smaller subset of data.

6 Full Fine-Tuning & The Memory Math

The Naive Approach

Full fine-tuning means updating **every single weight** in **every single matrix**. The training loop is straightforward:

1. **Forward:** Run input through model → get prediction
2. **Loss:** Calculate how wrong the prediction was (cross-entropy)
3. **Backward:** Compute the gradient (direction of needed change) for *every* parameter
4. **Update:** Nudge *every* parameter to reduce loss

Simple. Effective. The "obvious" approach. **But it has a catastrophic memory problem.**

Why Full Fine-Tuning Breaks

To run **inference** (just generating text), you only need the model weights in memory. But to **train**, your GPU must simultaneously hold:

Component	Memory Cost	Why
Model Weights	1x model size	The parameters themselves
Gradients	1x model size	One gradient value per parameter during backprop
Optimizer States	2x model size	AdamW stores momentum (<i>m</i>) and variance (<i>v</i>) for every parameter
Activations + Batch Data	~2-4x model size	Intermediate values stored for the backward pass
Total: ~6-8x		

Real-World Numbers

Using **fp16** (16-bit floating point = 2 bytes per parameter):

Model	Weight Size	VRAM to Train	Reality Check
SmolLM-135M	135M × 2B = 270 MB	~1-2 GB	✔ Fits on any GPU, even a laptop
LLaMA-3 8B	8B × 2B = 16 GB	~64-128 GB	⚠ Requires premium datacenter GPUs (A100 80GB barely fits)
LLaMA-3 70B	70B × 2B = 140 GB	~560+ GB	✗ Impossible on a single machine

[!CAUTION] **Context:** An NVIDIA A100 — the flagship datacenter GPU — has 80 GB of VRAM. Your laptop likely has 8-16 GB. You cannot full fine-tune even an 8B model on consumer hardware. We need a mathematical trick.

Beyond Memory: Quality Problems Too

Even if you *had* the memory, full fine-tuning has three additional failure modes:

Problem	What Happens
Overfitting	1,000 SEC filings (~5M tokens) vs 8 BILLION parameters = the model memorizes instead of learning.
Catastrophic Forgetting	Every weight gets updated, so general English knowledge gets overwritten. No "safe zone" of preserved knowledge.
Storage & Serving	Each fine-tuned variant = a full copy of all weights. 5 variants of LLaMA-70B = 5 × 140 GB = 700 GB of storage .

7 LoRA — The Breakthrough

We need a method that can:

- ☒ Train on a single GPU (or even a laptop)
- ☒ Not overfit on small datasets
- ☒ Preserve general knowledge
- ☒ Store just the changes, not the whole model
- ☒ Add no inference slowdown

The core question: *What if we could freeze the original model and only train a TINY set of new parameters?*

The Key Insight (Hu et al., 2021)

"When you fine-tune a large model, the weight *CHANGES* have very low intrinsic dimensionality."

Translation:

- The update matrix ΔW is **LOW-RANK**.
- Most of ΔW is redundant.
- You can represent the same update with far fewer numbers.

What Does "Low-Rank" Mean? (Intuition)

Rank	What It Means	Numbers Required
Full Rank (4096)	Every cell in a 4096×4096 matrix is unique and independent	16.7M numbers
Rank 1	Just one pattern, repeated and scaled across the entire matrix	8,192 numbers
Rank 32	32 independent patterns combined to approximate the full matrix	262K numbers

The empirical finding: Fine-tuning updates tend to be **rank 32-ish**, even for a 4096×4096 matrix.

🎵 The Choir Analogy

Imagine a choir of 100 people singing. But only **~4 are singing unique, independent parts**. The other 96 are just harmonizing — duplicating, doubling, and echoing. You don't need to record all 100 voices. Just capture the 4 unique voices and you can reconstruct the full sound.

LoRA does the same thing with weight updates.

The Mechanism

Instead of learning a full update matrix ΔW ($4096 \times 4096 =$ **16.7M parameters**), we decompose it into two much smaller matrices:

Original weight matrix W : $(4096 \times 4096) = 16.7\text{M params}$ ← FROZEN

Instead of learning ΔW directly:

$$\Delta W = B \times A$$

A : $(4096 \times 32) = 131\text{K params}$ "down-project to rank"

B : $(32 \times 4096) = 131\text{K params}$ "up-project back"

Total: 262K params vs 16.7M (64× fewer!)

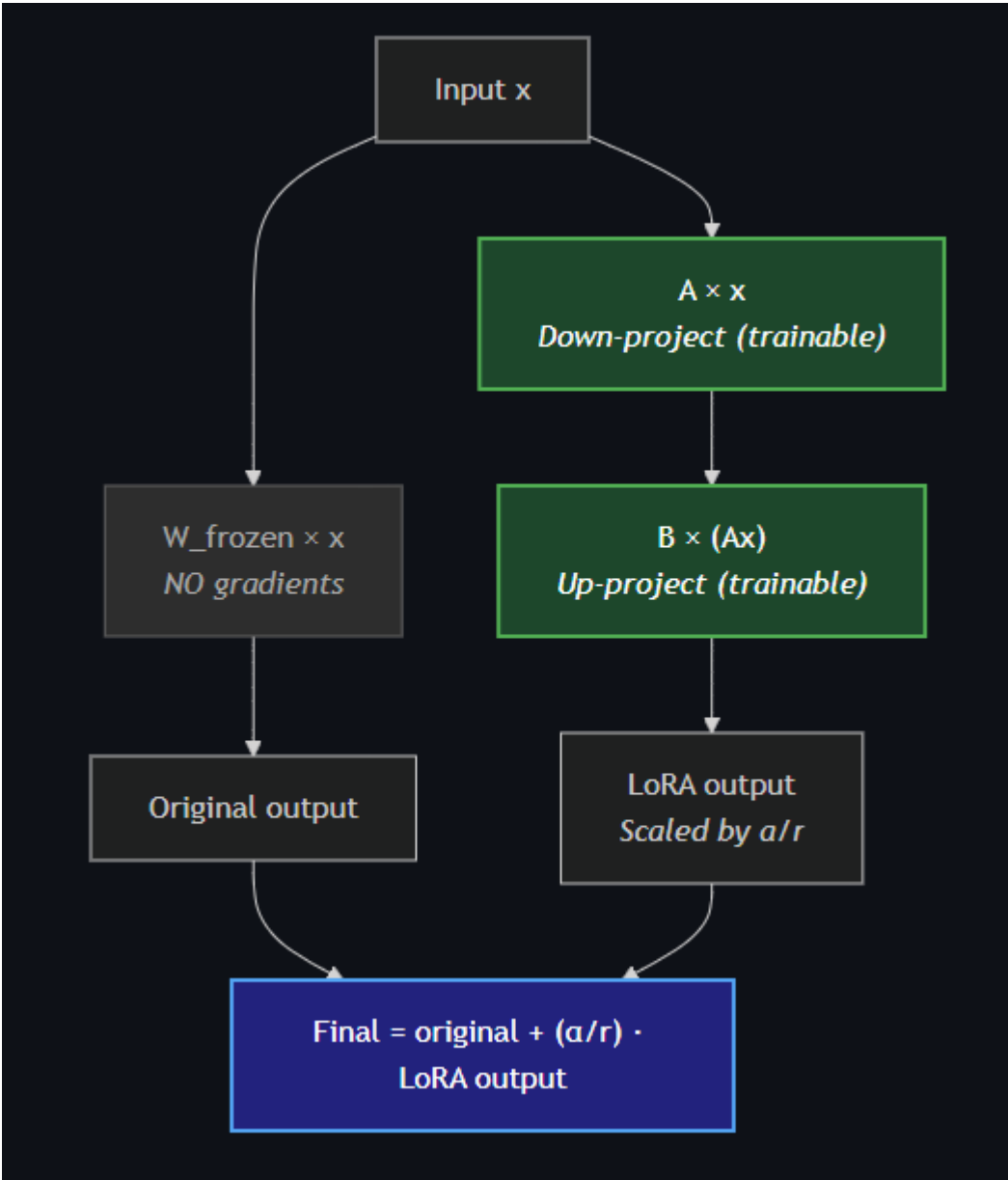
W is **FROZEN**. Only A and B are trained.

Final output: $w_{\text{frozen}} \cdot x + (\alpha/r) \cdot B \cdot A \cdot x$

Freeze the original. Train only the tiny adapters.

8 LoRA Visually & At Inference

During training, the input flows through **two parallel paths**:



The Magic of Merging

After training is complete, you can **merge** the adapters back into the original weights:

$$W_{\text{new}} = W + (\alpha/r) \cdot B \times A$$

This produces a **single matrix**. At inference time, the model runs at the **exact same speed** as the original — zero added latency, zero added computation. The LoRA adapters are gone; their learned patterns have been baked permanently into W .

9 The Memory Savings (Concrete) & LoRA Hyperparameters

Side-by-Side: LLaMA-3 8B

Full Fine-Tuning	LoRA (rank 32) + QLoRA
------------------	------------------------

	Full Fine-Tuning	LoRA (rank 32) + QLoRA
Weights	16 GB	16 GB → 4-bit → 4 GB
LoRA Params	—	~40 MB
Gradients	16 GB	~40 MB
Optimizer States	32 GB	~80 MB
Total VRAM	~ 64+ GB (needs A100)	~ 4.2 GB (fits on laptop!)
Saved Adapter File	~16 GB (full model copy)	~ 80 MB

5 customers, 5 fine-tunes? That's 400 MB of adapters sharing one base model, vs 80 GB of full model copies.

Key LoRA Hyperparameters

Parameter	What It Controls	Guidance
r (rank)	Size of the bottleneck. Higher = more capacity, more memory.	Common values: 8, 16, 32 , 64. Use 32 for CPT (learning new knowledge requires more capacity).
α (alpha)	Scaling factor that controls how much LoRA affects the final output. The effective scaling is α/r .	Rule of thumb: set $\alpha = r$ (so effective scaling = 1).
target_modules	Which weight matrices to attach LoRA adapters to.	Minimum: <code>q_proj</code> , <code>v_proj</code> . Recommended: All attention + all FFN layers.
dropout	Regularization inside LoRA.	Usually 0 (Unsloth optimizes for this).

[!WARNING] **CPT-Specific Gotcha:** For Continued Pre-Training, you **MUST** also target `embed_tokens` and `lm_head`!

Why? SFT teaches the model to use *existing* words differently — the vocabulary doesn't change. But CPT teaches the model entirely **new domain words** ("EBITDA", "10-K", "diluted EPS", "subordinated debentures"). The embedding layer needs to learn what these tokens *mean*, and the LM head needs to learn to *output* them.

The most common CPT mistake: copying an SFT LoRA config and wondering why domain adaptation doesn't work.

10 Quantization & QLoRA

Quantization compresses the **frozen** base weights from full precision down to fewer bits, dramatically reducing memory while keeping the trainable LoRA adapters in full precision.

Precision	Bits/Param	SmolLM-135M Size	Relative
-----------	------------	------------------	----------

Precision	Bits/Param	SmolLM-135M Size	Relative
Float32	32 bits → 4B/param	540 MB	1.0x
Float16	16 bits → 2B/param	270 MB	0.5x
Int8	8 bits → 1B/param	135 MB	0.25x
NF4	4 bits → 0.5B/param	67 MB	0.125x

QLoRA = Quantized base (4-bit) + LoRA adapters (16-bit)

The frozen weights that don't receive gradients are aggressively compressed. The tiny LoRA adapters that *do* receive gradients are kept in full precision to maintain training quality.

[!NOTE] **NF4 (Normal Float 4-bit)** is specifically designed for neural network weight distributions. Unlike naive 4-bit integer quantization, NF4 accounts for the bell-curve distribution of neural network weights, resulting in **almost no quality loss** despite 8x compression.

11 What Unsloth Actually Does

Unsloth is NOT a different training algorithm. It's the same math, executed wildly faster with engineering optimizations. Think of it like **NumPy vs hand-written Python matrix multiplication** — same math, wildly different speed.

Unsloth Optimization	What It Does	Impact
Custom CUDA Kernels	Rewritten attention + LoRA operations fused into single GPU operations	~2x speed
Fused Operations	Fewer memory round-trips between GPU memory and compute units	Less VRAM overhead
Smart Gradient Checkpointing	Instead of storing ALL layer activations for backprop, stores only some and recomputes the rest when needed	~50% less VRAM
Automatic Mixed Precision	Uses bf16/fp16 where safe, full precision only where needed	Memory + speed gains
Optimized Packing	No wasted padding tokens (see below)	More data per batch

Gradient Checkpointing Explained

Normal Training	Gradient Checkpointing
Layer 1 output → [STORED]	Layer 1 output → [STORED]
Layer 2 output → [STORED]	Layer 2 output → [RECOMPUTE when needed]
Layer 3 output → [STORED]	Layer 3 output → [STORED]

Normal Training	Gradient Checkpointing
...	Layer 4 output → [RECOMPUTE when needed]
Layer 24 output → [STORED]	...
Memory grows with depth! ~50% less VRAM, ~30% slower	

Unsloth's version is smarter about *which* layers to checkpoint, minimizing the recomputation penalty.

This is what `use_gradient_checkpointing="unsloth"` does in the config.

12 The Training Loop Internals

Regardless of whether you're doing pre-training, CPT, or SFT, the training loop is fundamentally **identical**. The only thing that changes is the **data**.

1. TOKENIZE	text → token IDs
2. FORWARD	tokens through model → logits (predictions)
3. LOSS	cross-entropy between predicted and actual next token
4. BACKWARD	compute gradients (only for LoRA params!)
5. UPDATE	optimizer step (AdamW) on LoRA params only
6. REPEAT	

Same loop for pre-training, CPT, and SFT. The **ONLY** difference is the **DATA**.

Key Training Concepts

Concept	What It Does	Example
Packing	Concatenates short texts into one dense sequence, eliminating wasted [PAD] tokens.	Without: [text1 + PAD PAD PAD]. With: [text1 text2 text3] ← no waste.
Gradient Accumulation	Simulates larger effective batch sizes when GPU memory is limited.	Batch size 16, accumulation steps 2 = effective batch 32
Warmup	Starts with a tiny learning rate and ramps up gradually.	Prevents early training from making destructively large updates to weights.
Early Stopping	Monitors eval loss; stops training when it stops improving.	Prevents overfitting on small datasets. The eval loss is your north star.

13 The Enemy — Catastrophic Forgetting

What Happens When You Only Train on Domain Data

Gradient updates are **GREEDY**. At each training step, the optimizer's sole objective is: *reduce loss on the CURRENT batch*. It has no mechanism to "remember" previously learned capabilities.

Before CPT:

"The cat sat on the" → "mat"

☒ general English

"Revenue increased by" → random garbage

☒ no finance

After CPT (100% domain data, no mixing):

"Revenue increased by" → "12% year over year"

☒ finance!

"The cat sat on the" → "balance sheet"

☒ forgot English!

The model doesn't "choose" to forget. The optimizer just never gets a signal to remember.

If every batch is a financial document:

- Gradients always point toward "be better at finance"
- Gradients **never** point toward "stay good at general text"
- General knowledge slowly erodes

5 Solutions to Forgetting

#	Solution	How It Works	Typical Values
1	Data Mixing	Mix domain data with general text (e.g., Wikipedia) so the optimizer sees both signals.	80% domain + 20% general
2	Low Learning Rate	Don't move weights too far from the pre-trained values.	1e-5 for full FT, 2e-4 for LoRA
3	LoRA Itself	Freezing 99% of base weights is an architectural guardrail against forgetting! The general knowledge lives in the frozen weights.	—
4	Short Training	Watch eval loss — when it plateaus, stop. More training only overfits.	1-3 epochs typically
5	Embedding LR	Use a 10x lower learning rate for <code>embed_tokens/lm_head</code> . These layers affect ALL tokens (not just domain ones), so large updates there destabilize everything.	2e-5 when main LR is 2e-4

14 Hands-On — CPT with SEC 10-K Filings

What Are 10-K Filings?

10-K = the **annual report every public US company files with the SEC**. They contain:

- Business descriptions and risk factors
- Complete financial statements
- Management discussion & analysis (MD&A)

The language is dense, formal, and jargon-heavy:

"The Company recognized impairment charges of \$142 million related to goodwill associated with the North America reporting unit during the fiscal year ended March 31, 2024."

SmolLM-135M has never seen this kind of text at scale. Time to teach it.

The 7-Step Plan

Step	Action	Goal
1	Load SmolLM-135M → try financial text → watch it fail	Establish the baseline
2	Get SEC 10-K filings from HuggingFace (Pl e I As/ SEC)	Acquire domain data
3	Clean & chunk the data	Prepare training input
4	CPT with Unsloth + LoRA	Train the model
5	Try financial text again → see the improvement	Validate domain adaptation
6	Measure: perplexity before vs after	Quantify the improvement
7	Experiment: with vs without data mixing	Study catastrophic forgetting

Step 1: Base Model Fails on Financial Text

```
from unsloth import FastLanguageModel

BASE_MODEL = "HuggingFaceTB/SmolLM-135M"
model, tokenizer = FastLanguageModel.from_pretrained(
    model_name=BASE_MODEL,
    max_seq_length=512,
    load_in_4bit=True,
)
```

Prompt: "The company reported total revenue of" **Base SmolLM:** "\$65 million, with a gross margin at 18 percent. Nordvåg's revenues are much higher than the combined sales for..."
← Generic, confused, not financial language.

Prompt: "Risk factors include exposure to fluctuations in" **Base SmolLM:** "energy prices, as well as increased stress levels due to family or work responsibilities. These types of stressors can affect mental health..." ← Drifted into health advice!

Step 2-3: Data Preparation Pipeline

Raw 10-K filing (18,000+ words)
|

- └─ Strip boilerplate (legal disclaimers, page numbers)
- └─ Remove reference/exhibit sections
- └─ Word-level chunking (256 words, 20% overlap)

↓

JSONL: {"text": "chunk of financial text..."}

Train/Val split: 80 filings train, 20 held out for evaluation

Result: 6,830 train chunks, 1,191 val chunks

Critical: split at the **filing level**, not the chunk level, to prevent data leakage.

Step 4: The LoRA + Training Config (Fully Annotated)

```
# LoRA Config – CPT-specific
model = FastLanguageModel.get_peft_model(
    model,
    r=32,                                # Higher rank for CPT (learning new
knowledge)
    target_modules=[
        "q_proj", "k_proj", "v_proj",    # Attention matrices
        "o_proj",                        # Output projection
        "gate_proj", "up_proj",          # FFN layers (knowledge lives here!)
        "down_proj",
        "embed_tokens", "lm_head",      # CPT-specific: new vocabulary!
    ],
    lora_alpha=32,                        # = rank, so scaling factor = 1
    lora_dropout=0,                      # Unsloth optimizes for this
    use_gradient_checkpointing="unsloth", # Smart checkpointing (~50% VRAM saved)
)

# Training Config
training_args = UnslothTrainingArguments(
    learning_rate=2e-4,                  # Standard for LoRA
    embedding_learning_rate=2e-5,        # 10× lower for embeddings!
    per_device_train_batch_size=16,      # Sequences per GPU per step
    gradient_accumulation_steps=2,       # Effective batch = 16 × 2 = 32
    num_train_epochs=2,                 # 2 epochs for CPT demo
    warmup_steps=50,                    # Gentle start
    packing=True,                       # No wasted padding
    lr_scheduler_type="cosine",          # Smooth decay to near-zero
    optim="adamw_8bit",                 # 8-bit AdamW saves VRAM
    eval_strategy="steps",
    eval_steps=25,                      # Check every 25 steps
    load_best_model_at_end=True,         # Auto-select best checkpoint
)
```

Actual Unsloth output at training start:

```
Unsloth: Setting lr = 2.00e-05 instead of 2.00e-04 for embed_tokens.  
Trainable parameters = 39,671,808 of 202,498,368 (19.59% trained)
```

Target ALL linear layers including FFN — that's where factual knowledge is stored.

Step 5-6: Results — Before vs After

Prompt: "The company reported total revenue of"

Output	
Base SmolLM	"\$65 million, with a gross margin at 18 percent. Nordvåg's revenues are much higher..."
After CPT	"\$15,074 in 1986 and net income for the years ended December 31, 1982. The Company also recorded a loss on its investment properties..."

The post-CPT output reads like actual SEC filing language — with fiscal year references, line-item terminology, and proper financial syntax.

Training Metrics: What to Watch

☒ Good Training Run	✗ Bad Signs
Train loss steadily decreasing	Eval loss going UP while train loss drops → Overfitting
Eval loss tracking train loss (not diverging)	Train loss spiking → Learning rate too high
Eval loss plateaus → stop (early stopping)	Loss stuck flat → LR too low or data too small

Key Definition:

- **Perplexity = $e^{(\text{loss})}$** . Lower = better. It measures how "surprised" the model is by the text.
- Base model perplexity on SEC validation chunks: **19.1**
- After CPT: **13.2** (a **31% improvement** — the model is significantly less surprised by financial text!)

15 The Forgetting Experiment

To empirically quantify catastrophic forgetting, the notebook runs two separate training runs:

	Run A: Pure Domain	Run B: Data Mixing
Training Data	100% SEC filings	80% SEC + 20% general text (WikiText)
Expected SEC Performance	☒ Great	Slightly worse
Expected General Performance	✗ Degraded (forgetting)	☒ Maintained

Evaluate on:

- SEC test set (domain performance)

- General English text (WikiText and similar)

Actual notebook output after pure CPT:

Prompt: "The cat sat on the" **Output:** "chair. The cat was released by the owner of the property and her name is entered in the inventory..." ← SEC filing language leaked into everyday English!

Prompt: "Once upon a time there was a" **Output:** "great deal of uncertainty about the future. The Company's investment in its own securities has been discontinued..." ← A fairy tale that turned into a financial report.

General English perplexity after CPT: 32.2 (higher than expected = the model *has* forgotten some general English). LoRA helps prevent this (base weights are frozen), but it's not perfect.

In production, you almost always want data mixing. The domain performance trade-off is small; the general capability preservation is massive.

NEFTune — A Subtle Trick

For **full** fine-tuning (not LoRA), there's an additional regularization technique:

```
# NEFTune: Add random noise to embeddings during training
input_embeds = input_embeds + α * random_noise
```

It acts as regularization (like dropout but for embeddings), prevents the model from overfitting to exact token sequences, and shows consistent improvements especially on small datasets.

[!NOTE] NEFTune is used in **full training mode**, not LoRA. LoRA already regularizes by constraining the update space to low-rank matrices.

16 The Limits of CPT — Smarter, Not Helpful

Here's the crucial insight. After CPT, we prompted the model like a chatbot:

Prompt: "Question: What was Apple's total revenue in fiscal year 2023?\nAnswer:"

CPT Model output: "The Company reported a \$15.6 million increase during the first quarter of 2023, reflecting an increased net income for each period since that time..."

Wait — why is this a failure? Because the model didn't *answer* the question. It just *continued the text* like a financial document. It auto-completed as if it were writing a 10-K filing, not conversing with a human.

[!CAUTION] **CPT teaches the model to SOUND like a domain. SFT teaches the model to RESPOND to instructions.**

We have trained the model on what it *knows* (CPT → domain language). Next session, we train it on what it *does* (SFT → instruction following with financial Q&A pairs from [virattt/financial-qa-10K](#)).

Today: what the model knows. **Next:** what it does. **Then:** what it values.

17 Key Takeaways

Takeaway

-
- | | |
|-------|---|
| 1 | Fine-tuning modifies the model itself — unlike prompting or RAG, where the weights stay frozen. |
| <hr/> | |
| 2 | LoRA: Freeze the base, train tiny adapter matrices (64× fewer params). This makes fine-tuning possible on a single GPU. |
| <hr/> | |
| 3 | CPT = same loss as pre-training , just with different (domain) data. The training loop is identical. |
| <hr/> | |
| 4 | For CPT: you MUST include <code>embed_tokens</code> + <code>lm_head</code> in LoRA targets — the model is learning new vocabulary. |
| <hr/> | |
| 5 | Catastrophic forgetting is real — mitigate it with data mixing (80/20), low learning rates, and LoRA itself (frozen base weights). |
| <hr/> | |
| 6 | Unloth = same math, faster execution — custom CUDA kernels, gradient checkpointing, packing. No algorithmic changes. |
| <hr/> | |
| 7 | CPT teaches domain language, NOT instruction following. Fine-tuning is not magic — it's gradient descent on different data. |
-

Fine-tuning is not magic — it's gradient descent on different data.

Made with ♥ by [Akshat Shethia](#)